

Transcrição do Material interativo: Comunicação entre Processos

Página 1

Processos em execução podem ou não se comunicar.

Página 2

Ou seja, processos podem ser independentes ou cooperantes. Processos cooperantes se comunicam e trocam informações. Mas por que dois processos cooperaram? Para compartilhar informações, aproveitar os recursos disponíveis para execução em paralelo e aumentar a velocidade da computação para modularizar as funções do programa onde cada uma é responsável por uma tarefa em específico e, além disso, conveniência no sentido de que o mesmo usuário pode usar diferentes aplicações ao mesmo tempo, e elas estarão executando em paralelo.

Página 3

Aqui temos dois modelos de comunicação: troca de mensagens e compartilhamento de memória. No modelo à esquerda, primitivas de *send* e *receive* são implementadas, e os processos enviam e recebem mensagens. No modelo à direita, há uma região de memória que é compartilhada entre os processos. Essa região de memória é utilizada para leitura e escrita de informação.

Página 4

Um problema clássico para demonstrar a comunicação entre processos é o produtor e o consumidor. Neste exemplo, há processos que inserem informações em um *buffer*, que pode ter um tamanho fixo ou ter um tamanho ilimitado, ou seja, produzem. E há processos que retiram informações do *buffer*, nesse caso, consomem.

Página 5

Temos aqui um exemplo de memória compartilhada com *buffer* de tamanho fixo. Temos a definição dos dados que são o *buffer*, a variável *in* e a variável *out*.

Página 6

No caso do produtor, ele só pode inserir elementos no *buffer* se existirem posições disponíveis. Então, o programa fica no *while* enquanto não puder produzir. Uma vez que houver pelo menos uma posição disponível, o produtor insere um valor no *buffer* e a variável *in* é atualizada, pois indica a próxima a posição que deve ser utilizada pelo produtor.

Página 7

Já no consumidor ocorre o contrário, pois ele fica preso no laço de repetição, ou seja, *while* enquanto o *buffer* estiver vazio. Uma vez que o *buffer* tiver ao menos um elemento, ele

poderá consumir e, com isso, atualizar o valor da variável *out*, que indica a posição do elemento que deve ser consumido.

Página 8

Além do compartilhamento de memória, que é o meio de comunicação entre *threads*, por exemplo, é possível usar o mecanismo de troca de mensagens. Neste caso, os processos que desejam se comunicar devem estabelecer um *link* de comunicação. Então, podemos pensar nas diferentes formas de implementar a comunicação, que pode ser direta, em que se sabe para qual processo a mensagem está sendo enviada e de qual processo a mensagem está sendo recebida. A comunicação também pode ser direta em portas de comunicação que são usadas como caixas postais. As mensagens não são endereçadas a um processo específico, nesse caso.

Página 9

Na implementação, a troca de mensagens pode ser bloqueante ou síncrona. Neste caso, na operação de envio da mensagem, o remetente fica bloqueado até que a mensagem seja recebida pelo destinatário. E na operação de recebimento, o destinatário fica bloqueado até que a mensagem fique disponível. Já na troca de mensagem assíncrona, não há bloqueio nem do lado do remetente nem do lado do receptor.

Página 10

Como exemplo de comunicação cliente-servidor, podemos implementar usando *Sockets*, que são estudados na disciplina de Redes; o *Pipes*, que é estudado aqui, em Sistemas Operacionais; e podemos usar também Chamadas de Procedimentos Remotas (RPC) quando temos que comunicar processos em um sistema distribuído.

Página 11

No caso de *Pipes*, o conceito é simples. Imagine uma tubulação onde, de um lado, a informação entra, e de outro lado, a informação sai. Em alguns sistemas operacionais como o sistema UNIX e Windows, há opção de implementar *Pipes* comuns e *Pipes* nomeados.

Página 12

No caso de *Pipes* comuns, a comunicação é unidirecional: escreve-se de um lado e lê-se do outro. É necessário uma relação de pai e filho entre os processos comunicantes.

Página 13

Em *Pipes* nomeados, a comunicação é bidirecional: tanto de um lado quanto de outro pode-se ler e escrever.

Chamadas e Procedimentos Remotas são usados em sistemas distribuídos. O RPC permite que sejam abstraídas as questões de implementação da comunicação, como *Sockets*, por exemplo. Ele fornece maneiras de chamar procedimentos que são executados do outro lado da comunicação. Para isso, define-se o que chamamos de *Stubs*, que são programas que executam tanto do lado do cliente quanto do lado do servidor para empacotar e desempacotar as informações trocadas, abstraindo para o programador questões relacionadas às arquiteturas das máquinas, por exemplo.